

NgRx Project — Step-by-Step with Code

Angular 17 · Standalone · No NgModule · Core Only (Store, Actions, Reducers, Selectors)

[What We Build](#)[Setup](#)[Folder Structure](#)[Step 1 — State](#)[Step 2 — Actions](#)[Step 3 — Reducer](#)[Step 4 — Selectors](#)[Step 5 — Register Store](#)[Step 6 — Routing](#)[Step 7 — Root Component](#)[Step 8 — main.ts](#)[Step 9 — Form Page](#)[Step 10 — Summary Page](#)[Step 11 — Details Page](#)[Full Data Flow Trace](#)[Common Mistakes](#)[Quick Reference](#)

What We Are Building

A 3-page Angular 17 app that demonstrates the complete NgRx Core flow:

Page	Route	What it does	NgRx role
 Profile Form	<code>/form</code>	User fills in name, email, age and clicks Save	Dispatches an Action to the Store
 Summary	<code>/summary</code>	Displays name + email only	Reads two Selectors from the Store
 Details	<code>/details</code>	Displays all 3 fields in a table, can also clear	Reads full profile Selector, Dispatches clear Action

The Summary and Details pages receive their data **directly from the NgRx Store** — no service injection, no `@Input()`, no prop drilling. They just select what they need.

Project Setup (Windows · Node 18+)

Before You Start — Verify Your Environment

Open **Command Prompt** or **PowerShell** and run:

```
# Check Node — must be 18 or higher
node -v
# Expected: v18.19.0 or v20.x.x

# Check npm — comes with Node
npm -v
# Expected: 10.x.x
```

If `node -v` throws an error, Node is not on your PATH. Re-run the Node installer and tick **"Add to PATH"** during installation.

Step A — Install the Angular CLI

Install once globally — reuse for all future Angular projects:

```
npm install -g @angular/cli@17
```

Verify:

```
ng version
# Should show: Angular CLI: 17.x.x
```

If you get *"ng is not recognized"* — close the terminal, re-open it, and try again. Windows needs a fresh session to pick up newly installed global packages.

Step B — Create the Project

```
# Navigate to where you want the project
cd C:\Users\YourName\projects

# Create the Angular 17 app
# --standalone → uses standalone components, no NgModule
# --routing → generates app.routes.ts automatically
# --style=css → use plain CSS
ng new ngrx-user-profile --standalone --routing --style=css
```

It will take 1–3 minutes to install all packages.

Output:

✓ Packages installed successfully.
Successfully initialized git.

Step C — Install NgRx

```
cd ngrx-user-profile
npm install @ngrx/store@17
```

Step D — Generate Components & Create State Files

```
# Create the state folder
mkdir src\app\state

# Generate the 3 page components (standalone, no test files)
ng generate component components/profile-form --standalone --skip-tests
ng generate component components/profile-summary --standalone --skip-tests
ng generate component components/profile-details --standalone --skip-tests

# Create the 4 NgRx state files (Command Prompt)
type nul > src\app\state\user-profile.state.ts
```

```
type nul > src\app\state\user-profile.actions.ts
type nul > src\app\state\user-profile.reducer.ts
type nul > src\app\state\user-profile.selectors.ts
```

PowerShell users: replace `type nul >` with `New-Item`

Example: `New-Item src\app\state\user-profile.state.ts`

Step E — Confirm the App Runs (Before Writing Any NgRx)

```
ng serve
```

Open `http://localhost:4200`. You should see the default Angular welcome page. If yes — your setup is correct. Press `Ctrl` + `C` to stop.

Final Folder Structure

After all the setup steps, your `src/app/` folder should look like this:

```
src/
└─ app/
    └─ state/
        └─ user-profile.state.ts
        └─ user-profile.actions.ts
        └─ user-profile.reducer.ts
        └─ user-profile.selectors.ts
    └─ components/
        └─ profile-form/
            └─ profile-form.component.ts
            └─ profile-form.component.html
        └─ profile-summary/
            └─ profile-summary.component.ts
            └─ profile-summary.component.html
        └─ profile-details/
            └─ profile-details.component.ts
            └─ profile-details.component.html
```

← All NgRx files live here

← Shape of our state (interface + initial value)

← What events can happen

← How state changes in response to events

← How components read slices of state

← Page 1: dispatches actions

← Page 2: reads store (name + email)

← Page 3: reads store (all fields)

app.component.ts	← Root component (nav + router-outlet)
app.component.html	
app.config.ts	← Where we register the Store
app.routes.ts	← Route definitions
main.ts	← Entry point

Now write the code — work through the steps below in order, top to bottom.

1 Define the State Shape

Concept: Before writing Actions or Reducers, define *what your state looks like*. This is a TypeScript interface (the shape) and a constant (the starting value). Every other NgRx file imports from here — it is the single source of truth for the state's structure.

src/app/state/user-profile.state.ts

```
// This interface describes the exact shape of the userProfile state slice.
// Think of it as the "schema" for this part of the store.
export interface UserProfileState {
  name: string;
  email: string;
  age: number | null; // null = not filled in yet
}

// The starting value when the app first loads.
// The reducer uses this as its default – before any action is dispatched.
export const initialUserProfileState: UserProfileState = {
  name: '',
  email: '',
  age: null,
};
```

Why a separate file? This interface is imported by the reducer, the selectors, and components. If you rename a field, TypeScript instantly flags every file that breaks — automatically. No manual search-and-replace needed.

2 Create Actions

Concept: Actions are the *only* way to tell the store something happened. They are plain objects with a `type` string and optional data (payload). `createAction()` gives us type-safe factory functions for producing them.

src/app/state/user-profile.actions.ts

```
import { createAction, props } from '@ngrx/store';

// — Action 1: Save the profile —————
//
// `createAction` returns a FACTORY FUNCTION – not the action object itself.
// You call the factory later (in a component) to produce the actual action.
//
// Naming convention: '[Source] Event Description'
// The source in brackets tells you WHERE this came from when you see it
// in DevTools among hundreds of other actions.
//
// `props<T>()` defines the payload shape with full TypeScript safety.
export const saveUserProfile = createAction(
  '[Profile Form] Save Profile',
  props<{ name: string; email: string; age: number | null }>()
);

// — Action 2: Clear the profile —————
//
// No `props()` – this action carries no data.
// It just signals "reset everything back to the initial state".
export const clearUserProfile = createAction(
  '[Profile Form] Clear Profile'
);
```

What does calling `saveUserProfile()` actually produce?

```
// You call the factory function like this in a component:
saveUserProfile({ name: 'Reza', email: 'reza@x.com', age: 25 })

// It produces this plain object – the actual Action:
{ type: '[Profile Form] Save Profile', name: 'Reza', email: 'reza@x.com', age: 25 }
```

Events, not commands. Notice the wording: "*Save Profile*" not "*Set Name*". Actions describe events that already happened — facts about the past. This mindset shift makes action logs readable and debuggable.

3 Create the Reducer

Concept: The Reducer is a *pure function* — it takes the current state and an action, and returns a **brand new state object**. It never mutates the old state. No side effects. No HTTP calls. Same inputs always produce the same output.

src/app/state/user-profile.reducer.ts

```
import { createReducer, on } from '@ngrx/store';
import { initialUserProfileState } from './user-profile.state';
import { saveUserProfile, clearUserProfile } from './user-profile.actions';

// `createReducer` is NgRx's modern, type-safe way to define a reducer.
// It replaces the old switch/case pattern from classic Redux.
//
// Arguments:
// 1st → The initial state (used on first load when state is undefined)
// 2nd+ → `on(action, handlerFn)` pairs – one for each action to handle
export const userProfileReducer = createReducer(

  // Starting state
  initialUserProfileState,

  // — Handle saveUserProfile —————
  // `on(action, (currentState, actionPayload) => newState)`
  //
  // The spread `...state` copies ALL existing fields first.
  // Then we list the fields to override with new values.
  // Result: a BRAND NEW object – the original `state` is never touched.
  on(saveUserProfile, (state, { name, email, age }) => ({
    ...state, // copy everything from old state
    name,     // override with new values from the action payload
    email,
    age,
  })),
```

```
// — Handle clearUserProfile —————  
// Return a fresh copy of the initial state – effectively wiping everything.  
on(clearUserProfile, () => ({ ...initialUserProfileState })))  
  
);
```

Why immutability matters:

NgRx detects changes by checking if the object *reference* changed. If you write

`state.name = 'Reza'` and return the same object, the reference is identical — NgRx thinks nothing happened and your UI won't update.

By returning `{ ...state, name: 'Reza' }` you get a new object with a new reference, so the store detects the change and notifies all selectors.

4 Create Selectors

Concept: Selectors are functions that read specific slices from the store. Define them once, reuse them in any component. They are *memoized* — they only recompute when the relevant part of state actually changed, preventing unnecessary re-renders.

src/app/state/user-profile.selectors.ts

```
import { createFeatureSelector, createSelector } from '@ngrx/store';  
import { UserProfileState } from './user-profile.state';  
  
// — Step 1: Feature Selector —————  
//  
// The full store is one big object: { userProfile: {...}, otherFeature: {...} }  
// `createFeatureSelector` grabs just OUR slice from the root store.  
//  
// IMPORTANT: The string 'userProfile' MUST exactly match  
//             the key used in provideStore() inside app.config.ts.  
export const selectUserProfileFeature =  
  createFeatureSelector<UserProfileState>('userProfile');  
  
// — Step 2: Property Selectors —————
```



```
//
// `createSelector` takes:
//   - One or more "input" selectors
//   - A "projector" function that receives their results and returns the data
//
// Memoization: if the input selector returns the same reference as last time,
// the projector is skipped entirely and the cached result is returned.

// Select just the name
export const selectUserName = createSelector(
  selectUserProfileFeature,          // input: grab the feature slice
  (userProfile) => userProfile.name // projector: extract just the name
);

// Select just the email
export const selectUserEmail = createSelector(
  selectUserProfileFeature,
  (userProfile) => userProfile.email
);

// Select just the age
export const selectUserAge = createSelector(
  selectUserProfileFeature,
  (userProfile) => userProfile.age
);

// Select the full profile object (when a component needs all 3 fields)
export const selectFullUserProfile = createSelector(
  selectUserProfileFeature,
  (userProfile) => userProfile
);

// — Bonus: Derived Selector —————
//
// Combine multiple selectors to produce a formatted string.
// This logic lives here – not scattered across 3 different components.
// Any component that needs "Reza (reza@x.com)" just uses this selector.
export const selectUserDisplayLine = createSelector(
  selectUserName,
  selectUserEmail,
  (name, email) => (name ? `${name} (${email})` : 'No profile saved yet')
);
```

Why not just pass the whole store to every component?

A component that only displays the user's name should not re-render when the email changes. Selectors give each component a precise, memoized subscription to exactly what it needs — nothing more.

5 Register the Store

Concept: In Angular 17 standalone apps there is no `AppModule`. Everything is registered through `app.config.ts` using provider functions. This is where we connect our reducer to Angular's DI system via `provideStore()`.

src/app/app.config.ts

```
import { ApplicationConfig } from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideStore } from '@ngrx/store'; // ← NgRx provider function

import { routes } from './app.routes';
import { userProfileReducer } from './state/user-profile.reducer';

export const appConfig: ApplicationConfig = {
  providers: [
    // Register the router
    provideRouter(routes),

    // Register the NgRx Store.
    // The object you pass defines the ROOT store shape.
    //
    // Key → the string used in createFeatureSelector() – MUST MATCH exactly
    // Value → the reducer that manages that slice
    provideStore({
      userProfile: userProfileReducer,

      // When your app grows, add more slices here:
      // products: productsReducer,
      // cart: cartReducer,
```

```
    }},  
  ],  
};
```

The key string must match. The string `'userProfile'` here and the string `'userProfile'` in `createFeatureSelector()` (Step 4) *must be identical*. A typo here produces no error — the selector just returns `undefined` silently. This is a common source of confusion for beginners.

6 Set Up Routing

src/app/app.routes.ts

```
import { Routes } from '@angular/router';  
import { ProfileFormComponent } from '../components/profile-form/profile-form.component';  
import { ProfileSummaryComponent } from '../components/profile-summary/profile-summary.component';  
import { ProfileDetailsComponent } from '../components/profile-details/profile-details.component';  
  
export const routes: Routes = [  
  // Redirect the empty URL to /form so the app has a sensible default landing page  
  { path: '', redirectTo: 'form', pathMatch: 'full' },  
  
  // Page 1 – the form (dispatches actions)  
  { path: 'form', component: ProfileFormComponent },  
  
  // Page 2 – shows name + email (reads store)  
  { path: 'summary', component: ProfileSummaryComponent },  
  
  // Page 3 – shows all fields (reads store)  
  { path: 'details', component: ProfileDetailsComponent },  
];
```

7 Root Component — Nav + Router Outlet

The root component provides the navigation bar and the `<router-outlet>` where each page component renders.

src/app/app.component.ts

```
import { Component } from '@angular/core';
import { RouterOutlet, RouterLink, RouterLinkActive } from '@angular/router';

@Component({
  selector: 'app-root',
  standalone: true,
  // In standalone components, directives/pipes are imported DIRECTLY here.
  // There is no NgModule to declare them in.
  imports: [RouterOutlet, RouterLink, RouterLinkActive],
  templateUrl: './app.component.html',
})
export class AppComponent {
  title = 'NgRx User Profile App';
}
```

src/app/app.component.html

```
<!-- Navigation bar with links to all 3 pages -->
<nav style="display:flex; gap:1rem; padding:1rem; background:#1a1a2e; color:white;">
  <strong>NgRx App</strong>

  <!-- routerLinkActive adds the 'active-link' CSS class when this route is active -->
  <a routerLink="/form" routerLinkActive="active-link">📝 Edit Profile</a>
  <a routerLink="/summary" routerLinkActive="active-link">👤 Summary</a>
  <a routerLink="/details" routerLinkActive="active-link">📄 Details</a>
</nav>

<!-- RouterOutlet renders whichever component matches the current URL -->
<main style="padding:2rem;">
  <router-outlet />
</main>

<style>
```

```
a { color: #aaa; text-decoration: none; }
a.active-link { color: white; font-weight: bold; border-bottom: 2px solid #e94560; }
</style>
```

8 Entry Point

In Angular 17 standalone apps, `main.ts` uses `bootstrapApplication()` instead of the old `platformBrowserDynamic().bootstrapModule(AppModule)`. This file is mostly generated for you — just confirm it looks like this:

src/main.ts

```
import { bootstrapApplication } from '@angular/platform-browser';
import { appConfig } from './app/app.config';
import { AppComponent } from './app/app.component';

// bootstrapApplication takes:
// 1. The root component (AppComponent)
// 2. The app config (which carries our providers: router, store, etc.)
bootstrapApplication(AppComponent, appConfig)
  .catch((err) => console.error(err));
```

9 Page 1 — Profile Form Component

This component's NgRx role: Dispatcher.

It does NOT directly change state. It describes what happened by dispatching an Action. The Reducer decides what to do with it. The component never touches the state directly.

src/app/components/profile-form/profile-form.component.ts

```
import { Component, OnInit } from '@angular/core';
import { ReactiveFormsModule, FormBuilder, FormGroup, Validators } from '@angular/forms';
import { Store } from '@ngrx/store';
import { Router } from '@angular/router';
import { Observable } from 'rxjs';
```

```
import { AsyncPipe, NgIf } from '@angular/common';

// Import OUR actions and selectors
import { saveUserProfile, clearUserProfile } from '../../state/user-profile.actions';
import { selectFullUserProfile } from '../../state/user-profile.selectors';
import { UserProfileState } from '../../state/user-profile.state';

@Component({
  selector: 'app-profile-form',
  standalone: true,
  imports: [
    ReactiveFormsModule, // required for [formGroup] and formControlName directives
    AsyncPipe,           // required for `| async` in the template
    NgIf,                // required for *ngIf
  ],
  templateUrl: './profile-form.component.html',
})
export class ProfileFormComponent implements OnInit {

  profileForm!: FormGroup;
  savedProfile$: Observable<UserProfileState>;

  constructor(
    private fb: FormBuilder,
    private store: Store, // ← inject the NgRx Store like any Angular service
    private router: Router
  ) {}

  ngOnInit(): void {
    // Build the reactive form
    this.profileForm = this.fb.group({
      name: ['', [Validators.required, Validators.minLength(2)]],
      email: ['', [Validators.required, Validators.email]],
      age: [null, [Validators.required, Validators.min(1), Validators.max(120)]],
    });

    // SELECT the current profile from the store.
    // store.select() returns an Observable – it emits whenever this slice changes.
    this.savedProfile$ = this.store.select(selectFullUserProfile);

    // Pre-fill the form if data already exists in the store
    // (e.g. the user navigated away and came back)
    this.savedProfile$.subscribe((profile) => {
      if (profile.name) {
        this.profileForm.patchValue({
          name: profile.name,
          email: profile.email,
        });
      }
    });
  }
}
```

```

        age: profile.age,
      });
    }
  });
}

onSubmit(): void {
  if (this.profileForm.valid) {
    const { name, email, age } = this.profileForm.value;

    // DISPATCH the action.
    // store.dispatch() sends the action through the NgRx pipeline.
    // The reducer handles it → store updates → selectors emit → UI re-renders.
    this.store.dispatch(saveUserProfile({ name, email, age }));

    // Navigate to the summary page after saving
    this.router.navigate(['/summary']);
  }
}

onClear(): void {
  this.store.dispatch(clearUserProfile()); // dispatch clear – no payload needed
  this.profileForm.reset();               // also reset the form UI
}

// Convenience getters – cleaner access in the template
get nameCtrl() { return this.profileForm.get('name')!; }
get emailCtrl() { return this.profileForm.get('email')!; }
get ageCtrl() { return this.profileForm.get('age')!; }
}

```

src/app/components/profile-form/profile-form.component.html

```

<div style="max-width:480px;">
  <h2>Edit User Profile</h2>

  <!-- [formGroup] binds this <form> to the profileForm instance in the component -->
  <form [formGroup]="profileForm" (ngSubmit)="onSubmit()">

    <!-- Name field -->
    <div style="margin-bottom:1rem;">
      <label for="name">Full Name</label><br />
      <input id="name" type="text" formControlName="name" placeholder="Enter name" />
      <!-- Show error only if the field was touched AND is invalid -->
      <div *ngIf="nameCtrl.touched && nameCtrl.invalid" style="color:red;font-size:0.85rem;">

```

```

    <span *ngIf="nameCtrl.errors?.['required']">Name is required.</span>
    <span *ngIf="nameCtrl.errors?.['minlength']">At least 2 characters.</span>
  </div>
</div>

<!-- Email field -->
<div style="margin-bottom:1rem;">
  <label for="email">Email Address</label><br />
  <input id="email" type="email" formControlName="email" placeholder="Enter email" />
  <div *ngIf="emailCtrl.touched && emailCtrl.invalid" style="color:red;font-size:0.85rem;">
    <span *ngIf="emailCtrl.errors?.['required']">Email is required.</span>
    <span *ngIf="emailCtrl.errors?.['email']">Enter a valid email address.</span>
  </div>
</div>

<!-- Age field -->
<div style="margin-bottom:1.5rem;">
  <label for="age">Age</label><br />
  <input id="age" type="number" formControlName="age" placeholder="Enter age" />
  <div *ngIf="ageCtrl.touched && ageCtrl.invalid" style="color:red;font-size:0.85rem;">
    <span *ngIf="ageCtrl.errors?.['required']">Age is required.</span>
    <span *ngIf="ageCtrl.errors?.['min'] || ageCtrl.errors?.['max']">Age must be 1-120.</span>
  </div>
</div>

<!-- Submit button – disabled while form is invalid -->
<button type="submit" [disabled]="profileForm.invalid">💾 Save to Store</button>
  &nbsp;
  <button type="button" (click)="onClear()">🗑️ Clear</button>

</form>

<!-- DEBUG VIEW: shows the current store state live (remove in production) -->
<hr style="margin-top:2rem;" />
<h4>📦 Current Store State (Debug View):</h4>
<pre *ngIf="savedProfile$ | async as profile">{{ profile | json }}</pre>

</div>

```


10 Page 2 — Profile Summary Component

This component's NgRx role: Reader only.

It never imports actions or the reducer. It only selects the data it needs from the store. It knows nothing about how the data got there.

src/app/components/profile-summary/profile-summary.component.ts

```
import { Component } from '@angular/core';
import { Store } from '@ngrx/store';
import { Observable } from 'rxjs';
import { AsyncPipe, NgIf } from '@angular/common';
import { RouterLink } from '@angular/router';

// This component only needs selectors – no actions imported here
import {
  selectUserName,
  selectUserEmail,
  selectUserDisplayLine
} from '../../../state/user-profile.selectors';

@Component({
  selector: 'app-profile-summary',
  standalone: true,
  imports: [AsyncPipe, NgIf, RouterLink],
  templateUrl: './profile-summary.component.html',
})
export class ProfileSummaryComponent {

  // Each property is an Observable.
  // It emits a new value automatically whenever the store slice it watches changes.
  // We use `| async` in the template – it subscribes AND unsubscribes automatically.
  name$: Observable<string>;
  email$: Observable<string>;
  displayLine$: Observable<string>;

  constructor(private store: Store) {
    // store.select(selector) returns an Observable for that selector
    this.name$ = this.store.select(selectUserName);
    this.email$ = this.store.select(selectUserEmail);
  }
}
```

```

    this.displayLine$ = this.store.select(selectUserDisplayLine);
  }
}

```

src/app/components/profile-summary/profile-summary.component.html

```

<div style="max-width:480px;">
  <h2>Profile Summary</h2>
  <p style="color:gray;">Reads Name and Email from the NgRx Store. No @Input(). No service
  injection.</p>

  <div style="background:#f8f9fa; padding:1.5rem; border-radius:8px; margin-bottom:1rem;">

    <!-- `| async` subscribes to the Observable and unwraps its current value.
    Angular automatically unsubscribes when this component is destroyed.
    No ngOnDestroy needed. No manual subscription management. -->

    <p>
      <strong>Name:</strong>
      <span *ngIf="name$ | async as name; else empty">{{ name }}</span>
    </p>

    <p>
      <strong>Email:</strong>
      <span *ngIf="email$ | async as email; else empty">{{ email }}</span>
    </p>

    <!-- The derived selector formats the combined display string for us -->
    <p><strong>Display line:</strong> {{ displayLine$ | async }}</p>
  </div>

  <!-- Fallback template shown when value is empty -->
  <ng-template #empty>
    <span style="color:#aaa; font-style:italic;">Not filled in yet</span>
  </ng-template>

  <p *ngIf="(name$ | async) === ''">
    📄 <a routerLink="/form">Go fill in the form first!</a>
  </p>
</div>

```

11 Page 3 — Profile Details Component

This component's NgRx role: Reader + Dispatcher.

It reads the full profile using `selectFullUserProfile` and can also dispatch the `clearUserProfile` action from its own button. Any component can dispatch actions — not just the form.

`src/app/components/profile-details/profile-details.component.ts`

```
import { Component } from '@angular/core';
import { Store } from '@ngrx/store';
import { Observable } from 'rxjs';
import { AsyncPipe, NgIf, JsonPipe } from '@angular/common';
import { RouterLink } from '@angular/router';

import { selectFullUserProfile } from '../../../state/user-profile.selectors';
import { UserProfileState } from '../../../state/user-profile.state';
import { clearUserProfile } from '../../../state/user-profile.actions';

@Component({
  selector: 'app-profile-details',
  standalone: true,
  imports: [AsyncPipe, NgIf, JsonPipe, RouterLink],
  templateUrl: './profile-details.component.html',
})
export class ProfileDetailsComponent {

  // Select the FULL profile as a single Observable
  fullProfile$: Observable<UserProfileState>;

  constructor(private store: Store) {
    this.fullProfile$ = this.store.select(selectFullUserProfile);
  }

  // Dispatching is not limited to the form component.
  // ANY component can dispatch ANY action at any time.
  onClearFromDetailsPage(): void {
    this.store.dispatch(clearUserProfile());
  }
}
```

`src/app/components/profile-details/profile-details.component.html`

```

<div style="max-width:480px;">
  <h2>Profile Details</h2>
  <p style="color:gray;">Reads the full profile object from the NgRx Store.</p>

  <!--
  `*ngIf="fullProfile$ | async as profile"` does two things at once:
    1. Subscribes to the Observable via `| async`
    2. Assigns the emitted value to the local template variable `profile`
       so we can reference it multiple times inside this block
  -->
  <div *ngIf="fullProfile$ | async as profile">

    <table>
      <tr>
        <th style="text-align:left;">Field</th>
        <th style="text-align:left;">Value</th>
      </tr>
      <tr>
        <td><strong>Full Name</strong></td>
        <td>{{ profile.name || '-' }}</td>
      </tr>
      <tr>
        <td><strong>Email</strong></td>
        <td>{{ profile.email || '-' }}</td>
      </tr>
      <tr>
        <td><strong>Age</strong></td>
        <td>{{ profile.age !== null ? profile.age : '-' }}</td>
      </tr>
    </table>

    <div style="margin-top:1.5rem; display:flex; gap:1rem;">
      <a routerLink="/form"><button>✎ Edit Profile</button></a>
      <button (click)="onClearFromDetailsPage()">🗑 Clear Profile</button>
    </div>

    <!-- Raw state dump – useful for learning, remove in production -->
    <hr style="margin-top:2rem;" />
    <h4>📦 Raw Store Slice:</h4>
    <pre>{{ profile | json }}</pre>
  </div>

</div>

```

Complete Data Flow Trace

Let's trace **every single thing that happens** from the moment the user clicks "Save to Store" to the moment the Summary page re-renders with the new name:

1. User fills in the form
 - └ name: 'Reza', email: 'reza@x.com', age: 25
2. User clicks "Save to Store"
 - └ onSubmit() is called in ProfileFormComponent
3. Component DISPATCHES an Action:
 - └ this.store.dispatch(
 saveUserProfile({ name: 'Reza', email: 'reza@x.com', age: 25 })
)
 - └ Produces the action object:
 { type: '[Profile Form] Save Profile', name: 'Reza', email: 'reza@x.com', age: 25 }
4. NgRx sends the Action to the Reducer:
 - └ userProfileReducer(currentState, action)
 - └ The on(saveUserProfile, ...) handler matches
 - └ Returns NEW state: { name: 'Reza', email: 'reza@x.com', age: 25 }
 - └ Original state object is untouched
5. The Store replaces its internal state with the new object
6. All active Selectors re-evaluate:
 - └ selectUserName → emits 'Reza'
 - └ selectUserEmail → emits 'reza@x.com'
 - └ selectUserAge → emits 25
 - └ selectFullUserProfile → emits { name: 'Reza', email: 'reza@x.com', age: 25 }
 - └ selectUserDisplayLine → emits 'Reza (reza@x.com)'
7. All components subscribed via store.select() receive new values:
 - └ ProfileSummaryComponent → re-renders: shows "Reza" and "reza@x.com"
 - └ ProfileDetailsComponent → re-renders: table shows all 3 fields
8. Router navigates to /summary
 - └ User sees the updated data immediately

Notice what the component did NOT do:

It did not call a setter method. It did not push to a BehaviorSubject. It did not reach into the store and assign a value. It just described what happened (`saveUserProfile`) and let the NgRx pipeline handle everything else.

Common Mistakes to Avoid

✗ Mistake 1 — Mutating state in the reducer

```
// WRONG – mutates the existing object, reference doesn't change, UI won't update
on(saveUserProfile, (state, { name }) => {
  state.name = name;    // ← NEVER DO THIS
  return state;
})

// CORRECT – returns a brand new object
on(saveUserProfile, (state, { name }) => ({
  ...state,
  name,
}))
```

✗ Mistake 2 — Feature key mismatch

```
// In app.config.ts:
provideStore({ userProfile: userProfileReducer })
//           ^^^^^^^^^^^ this string is the key

// In selectors – must be IDENTICAL:
createFeatureSelector<UserProfileState>('userProfile')
//           ^^^^^^^^^^^ must match exactly

// A typo produces no error – the selector just returns undefined silently.
// Always copy-paste this string, never retype it.
```

✗ Mistake 3 — Manually subscribing instead of using AsyncPipe

```
// WRONG – you must remember to unsubscribe to avoid memory leaks
ngOnInit() {
  this.store.select(selectUserName).subscribe(name => {
    this.name = name; // stored in a component property
  });
  // If you forget to unsubscribe in ngOnDestroy → memory leak
}

// CORRECT – AsyncPipe subscribes AND unsubscribes automatically
// In the template: {{ name$ | async }}
// When the component is destroyed, Angular cleans up the subscription for you.
```

✗ Mistake 4 — Calling HTTP inside a reducer

```
// WRONG – reducers are pure functions, no side effects allowed
on(saveUserProfile, (state, action) => {
  this.http.post('/api/save', action); // ← NEVER DO THIS
  return { ...state };
})

// HTTP calls belong in NgRx Effects (the next topic after mastering Core).
```

Quick Reference Card

Concept	File	Job	Key API
State	user-profile.state.ts	Defines shape + initial value	TS interface + <code>const</code>
Actions	user-profile.actions.ts	Describes events that can happen	<code>createAction</code> , <code>props</code>
Reducer	user-profile.reducer.ts	Produces new state	<code>createReducer</code> , <code>on</code>

Concept	File	Job	Key API
Selectors	<code>user-profile.selectors.ts</code>	Reads slices for components	<code>createFeatureSelector</code> , <code>createSelector</code>
Register	<code>app.config.ts</code>	Wires reducer into Angular DI	<code>provideStore({ key: reducer })</code>
Dispatch	Any component	Sends action to store	<code>store.dispatch(action(...))</code>
Select	Any component	Reads Observable from store	<code>store.select(selector)</code>

Running the App

```

ng serve
# Open http://localhost:4200
# You are redirected to /form

# 1. Fill in the form → click "Save to Store"
#   → redirected to /summary – name and email appear
# 2. Click "Details" in nav
#   → all 3 fields shown in the table
# 3. Click "Clear Profile"
#   → all fields reset across ALL pages simultaneously
#   → this is the store as a single source of truth in action
  
```

What Comes Next

1. **NgRx DevTools** — install the Redux DevTools Chrome extension + `@ngrx/store-devtools` to get a full action log and time-travel debugging. Covered in `03-ngrx-devtools.html`.
2. **NgRx Effects** — for async operations (HTTP calls, localStorage). Effects listen for actions, perform async work, then dispatch new actions.
3. **NgRx Entity** — a utility for managing collections (arrays) of items without manual array manipulation.